



# 1) Dataset Load and Preprocessing

---

```
# =====  
# 1. Import Required Libraries  
# =====  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
import seaborn as sns
```

```
# =====  
# 2. Load and Explore Dataset  
# =====  
data = pd.read_csv("real_estate_price_size.csv")  
print("First 5 rows:")  
print(data.head())  
  
# Check for missing values  
print("\nMissing values before drop:")  
print(data.isnull().sum())  
  
# Drop any missing values (none in this dataset)  
data = data.dropna()  
print("\nMissing values after drop:")  
print(data.isnull().sum())  
  
# Extract features and target  
X = data['size'].values      # feature (size)  
Y = data['price'].values    # target (price)  
  
# Display Variable X and Y  
print("\nDisplay Variables X:")  
print(X)  
print("\nDisplay Variables Y:")  
print(Y)
```

[Show hidden output](#)

## ✓ Min-Max Normalization

We use **Min-Max Normalization** to scale the data into a fixed range, usually between 0 and 1.

The formula is:

$$[X_{\text{norm}}] = \frac{X - X_{\min}}{X_{\max} - X_{\min}}$$

where:

- $(X)$  = original value
- $(X_{\min})$  = minimum value in the feature
- $(X_{\max})$  = maximum value in the feature
- $(X_{\text{norm}})$  = normalized value

```
# =====
# 3. Normalize Feature (Min-Max Scaling)
# =====
X_min = X.min()
X_max = X.max()
X = (X - X_min) / (X_max - X_min)
print("\nNormalized feature (first 5):", X[:5])

# Convert to numpy arrays (already done, but ensure)
X = np.array(X)
Y = np.array(Y)

# Display Data
print("\nNumpy Array X:")
print(X)
print("\nNumpy Array Y:")
print(Y)
```

Normalized feature (first 5): [0.1198597 0.12949456 0.00553289 0.75215005 0.

Numpy Array X:

```
[0.1198597 0.12949456 0.00553289 0.75215005 0.58389592 0.07003434
0.06687898 0.10351786 0.14860284 0.15759928 0.42605448 1.
0.15759928 0.38854971 0.60260794 0.66040242 0.15467874 0.10580733
0.14772961 0.40213244 0.10343714 0.05132232 0.53398984 0.02834688
0.03379905 0.45756406 0.06687898 0.62692624 0.14772961 0.74599343
0.38934222 0.14772961 0.08669905 0.03379905 0.27722416 0.10509554
0.25203264 0.06640935 0.15096569 0.16036573 0.39786903 0.14860284
0.25203264 0.62692624 0.42605448 0.16036573 0.11259503 0.16036573
0.11259503 0.10075142 0.12309578 0.39786903 0.39786903 0.18207902
0.12309578 0.75515131 0.4052878 0.59005254 0.25543016 0.06561684
0.25566497 0.16051983 0.53264698 0.39786903 0.15096569 0.2548798
0.38854971 0.05140304 0.15096569 0.74599343 0.39786903 0.12009451
0.12949456 0.05140304 0.15096569 0.15096569 0.51638586 0.62692624
0.14860284 0.5201723 0.14772961 0.47153571 0.14772961 0.1198597
0.15096569 0.38854971 0.40260941 0.08945816 0.55562241
0.15759928 0.43426575 0.15759928 0.16036573 0.16036573 0.05140304
0.40923567 0.75215005 0.12367548 0.16550236]
```

Numpy Array Y:

```
[234314.144 228581.528 281626.336 401255.608 458674.256 245050.28
```

```

265129.064 175716.48 331101.344 218630.608 279555.096 494778.992
215472.104 418753.008 444192.008 440201.616 248337.6 234178.16
225451.984 299416.976 268125.08 171795.24 412569.472 183459.488
168047.264 362519.72 271793.312 406852.304 297760.44 368988.432
301635.728 225452.32 207742.248 191486.896 285223.176 302000.92
269225.92 233493.208 292965.216 245747.2 310045.712 217468.224
287350. 414682.648 293044.496 300061.48 204302.976 201778.048
257828.416 262423.504 225656.12 393069.76 258637.008 269523.056
255629.16 500681.128 320345.52 395242.096 330677.128 251332.592
251188.824 263311.696 359674.44 334938.872 302393.384 304587.272
355251.2 271726.752 294582.944 454512.76 276875.632 181587.576
298926.496 211724.096 228313.024 286161.6 382120.152 365863.936
251560.04 342988.456 180307.216 408637.816 190909.056 282683.544
303597.216 376253.808 154282.128 327252.112 211904.536 354512.112
251140.656 338078.168 298170.88 266684.248 262477.856 252460.4
310522.592 383635.568 225145.248 274922.856]

```

```

# =====
# 4. Shuffle and Split Data
# =====
np.random.seed(42) # for reproducibility
indices = np.random.permutation(len(X))

# Display Randomly Generated Indices
print("Randomly Generated Indices:")
print(indices)

# Creating Random X and Y data based on Indices
X = X[indices]
Y = Y[indices]

# Display Random X and Y data
print("\nData of X: ")
print(X)
print("\nData of Y: ")
print(Y)

# Define Split (70%,30%)
split_index = int(0.7 * len(X))

# Display split (70%,30%)
print("\nSplit (70%,30%):")
print(split_index)

# Splitting data into Test and Train Sets
X_train = X[:split_index]
X_test = X[split_index:]
Y_train = Y[:split_index]
Y_test = Y[split_index:]

# Display Train and Test data

```

```

print("\nTrain data of X:")
print(X_train)
print("\nTrain data of Y:")
print(Y_train)
print("\nTest data of X:")
print(X_test)
print("\nTest data of Y:")
print(Y_test)

# Display Train and Test data sizes
print(f"\nTraining samples: {len(X_train)}, Test samples: {len(X_test)}")

```

[Show hidden output](#)

## 2) Implement Linear Regression

### Hypothesis Function

For Simple Linear Regression:

$$[\hat{y} = m x + c]$$

Where:

- $m$  = slope (weight parameter)
- $c$  = intercept (bias)
- $x$  = input feature (size)
- $\hat{y}$  = predicted price

### Cost Function (Mean Squared Error)

$$[J(m, c) = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2]$$

We minimize this function using Gradient Descent.

### Gradient Descent Update Rules

$$[m := m - \alpha \frac{\partial J}{\partial m}]$$

$$[c := c - \alpha \frac{\partial J}{\partial c}]$$

Where:

- $\alpha$  = learning rate

- $\partial J / \partial m$  = gradient w.r.t slope
- $\partial J / \partial c$  = gradient w.r.t intercept

## ✓ Linear Regression model using Gradient Descent.

### Parameters:

learning\_rate : float - step size for gradient descent  
 iterations : int - number of iterations

### Attributes:

m, c : float - learned slope and intercept  
 cost\_history : list - cost value at each iteration

```
# =====
# 5. Define Linear Regression Class (Gradient Descent)
# =====
class LinearRegressionGD:
    def __init__(self, learning_rate=0.01, iterations=1000):
        self.lr = learning_rate
        self.iterations = iterations
        self.m = None          # slope
        self.c = None          # intercept
        self.cost_history = []

    def _hypothesis(self, X):
        """Internal prediction:  $\hat{y} = mX + c$ """
        return self.m * X + self.c

    def _compute_cost(self, X, y):
        """Mean Squared Error cost:  $J = 1/(2n) * \sum (y - \hat{y})^2$ """
        n = len(y)
        y_pred = self._hypothesis(X)
        cost = (1 / (2 * n)) * np.sum((y - y_pred) ** 2)
        return cost

    def fit(self, X, y):
        """
        Run gradient descent to learn parameters m and c.
        X, y : numpy arrays (1D) of training data.
        """
        n = len(y)
        # Initialize parameters
        self.m = 0.0
        self.c = 0.0
        self.cost_history = []
```

```

    for i in range(self.iterations):
        y_pred = self._hypothesis(X)

        # Compute gradients (derivatives of cost w.r.t. m and c)
        dm = (1 / n) * np.sum((y_pred - y) * X)
        dc = (1 / n) * np.sum(y_pred - y)

        # Update parameters using gradient descent
        self.m -= self.lr * dm
        self.c -= self.lr * dc

        # Record cost
        cost = self._compute_cost(X, y)
        self.cost_history.append(cost)

        # Optional: print progress every 100 iterations
        if i % 100 == 0:
            print(f"Iteration {i:4d} : cost = {cost:.2f}")
    def predict(self, X):
        """Predict using the learned line."""
        return self._hypothesis(X)

```

### 3) Training & Calculation

```

# =====
# 6. Train the Model
# =====
model = LinearRegressionGD(learning_rate=0.1, iterations=1000)
model.fit(X_train, Y_train)

print("\n" + "="*40)
print("FINAL RESULTS")
print("="*40)
print(f"Slope (m)          : {model.m:.4f}")
print(f"Intercept (c)       : {model.c:.4f}")
print(f"Final training cost : {model.cost_history[-1]:.2f}")

```

```

Iteration    0 : cost = 38532375775.17
Iteration  100 : cost = 1173724733.05
Iteration  200 : cost = 892708469.72
Iteration  300 : cost = 781470662.41
Iteration  400 : cost = 737438157.74
Iteration  500 : cost = 720008279.22
Iteration  600 : cost = 713108815.57
Iteration  700 : cost = 710377723.76
Iteration  800 : cost = 709296645.16
Iteration  900 : cost = 708868709.73

```

```
=====
FINAL RESULTS
=====
Slope (m)      : 299677.5255
Intercept (c)   : 212731.4750
Final training cost : 708700348.60
```

## 4) Evaluation

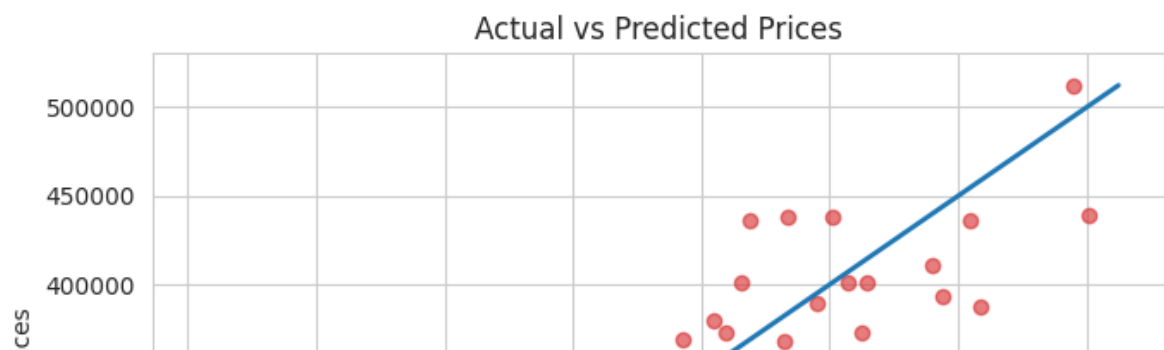
```
# =====
# 7. Evaluate on Test Set
# =====
test_cost = model._compute_cost(X_test, Y_test) # reuse internal method
print(f"Test cost          : {test_cost:.2f}")

Test cost          : 859111566.77
```

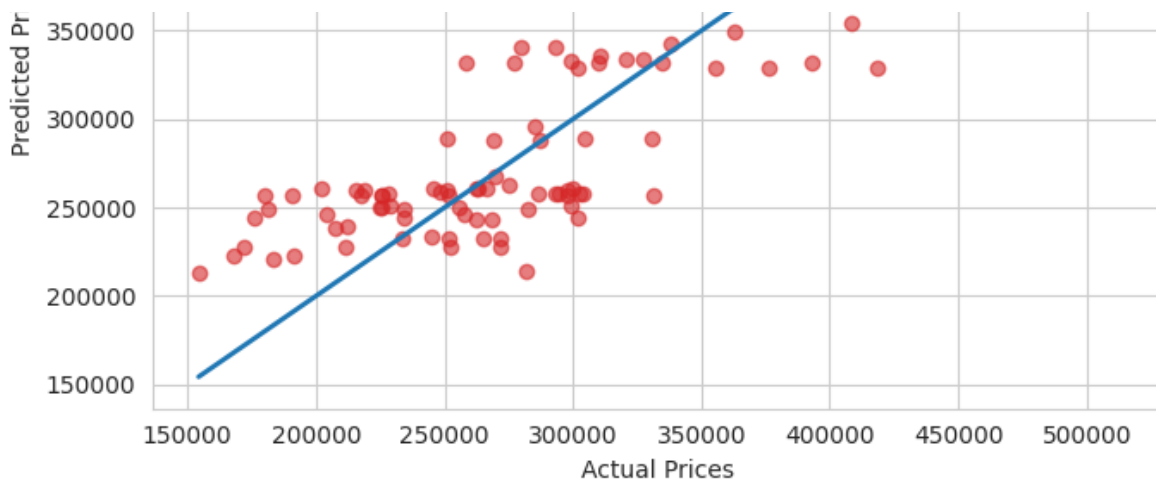
## 5) Visualization

```
# =====
# 8. Visualizations
# =====
sns.set_style("whitegrid")
```

```
# --- 8.1 Actual vs Predicted (full dataset) ---
Y_pred_full = model.predict(X)
plt.figure(figsize=(7,5))
plt.scatter(Y, Y_pred_full, color="#d62728", alpha=0.6)
min_val = min(Y.min(), Y_pred_full.min())
max_val = max(Y.max(), Y_pred_full.max())
plt.plot([min_val, max_val], [min_val, max_val], color="#1f77b4", lw=2)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted Prices")
plt.tight_layout()
plt.show()
```

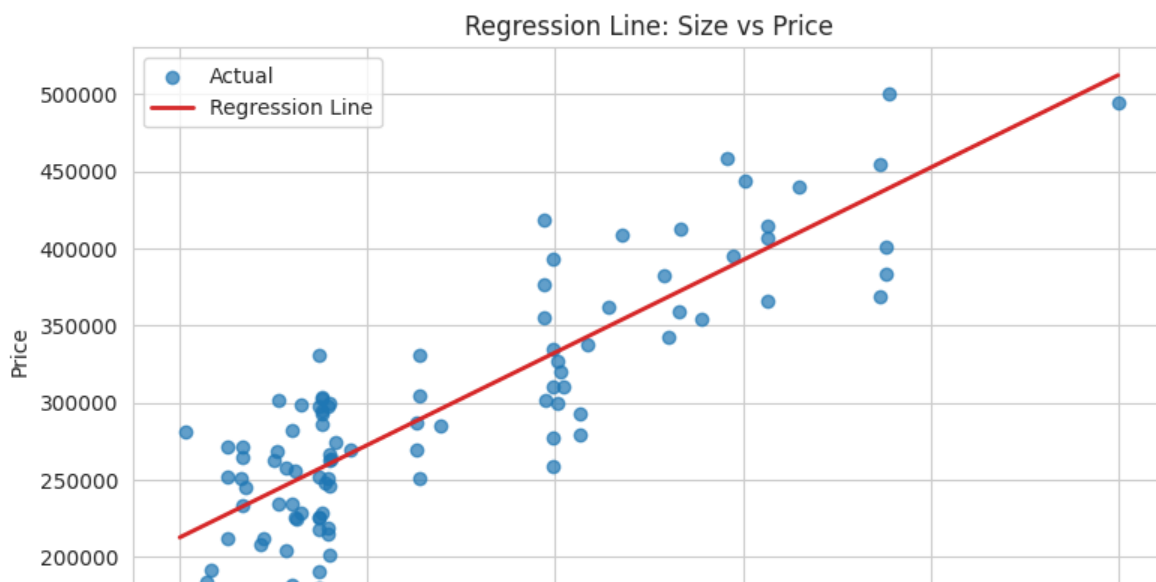


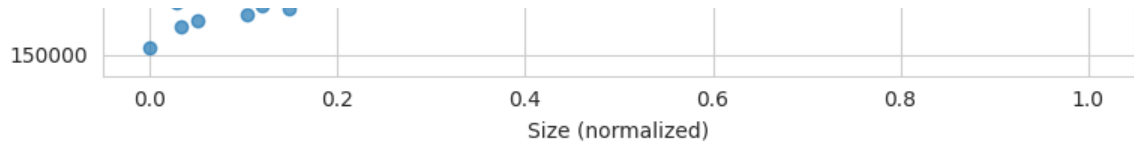




```
# --- 8.2 Regression Line (size vs price) ---
# Sort for clean line plotting
sorted_idx = np.argsort(X)
X_sorted = X[sorted_idx]
Y_sorted = Y[sorted_idx]
Y_pred_sorted = Y_pred_full[sorted_idx]

plt.figure(figsize=(8,5))
plt.scatter(X_sorted, Y_sorted, label="Actual", color="#1f77b4", alpha=0.7)
plt.plot(X_sorted, Y_pred_sorted, label="Regression Line", color="#d62728",
plt.xlabel("Size (normalized)")
plt.ylabel("Price")
plt.title("Regression Line: Size vs Price")
plt.legend()
plt.tight_layout()
plt.show()
```





```
# --- 8.3 Cost Reduction Over Iterations ---  
plt.figure(figsize=(8,4))  
plt.plot(model.cost_history, color='green')  
plt.xlabel("Iteration")  
plt.ylabel("Cost")  
plt.title("Cost Reduction during Gradient Descent")  
plt.grid(True)  
plt.tight_layout()  
plt.show()
```

